```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from xgboost import XGBRegressor
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error

# Load the datasets
try:
    train_df = pd.read_csv('/content/Training_set.csv')
    test_df = pd.read_csv('/content/Testing_set.csv')
    print("Datasets loaded successfully.")
except FileNotFoundError:
    print("Error: Make sure 'Training_set.csv' and 'Testing_set.csv' are uploaded to '/content/'.")
    exit()

# Assuming the last column is the target variable
X_train = train_df.iloc[:, :-1]
y_train = train_df.iloc[:, -1]
X_test = test_df.iloc[:, :-1]
y_test = test_df.iloc[:, -1]

# Feature Scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Function to train, predict, and evaluate models for both training and testing sets
def train_evaluate_model(model, model_name, X_train_data, y_train_data, X_test_data, y_test_data):
    print(f"\n--- {model_name} ---")
    model.fit(X_train_data, y_train_data)

    # Evaluate on Training Set
    y_train_pred = model.predict(X_train_data)
    rmse_train = np.sqrt(mean_squared_error(y_train_data, y_train_pred))
    r2_train = r2_score(y_train_data, y_train_pred)
    mae_train = mean_absolute_error(y_train_data, y_train_pred)
```

```
    mae_train = mean_absolute_error(y_train_data, y_train_pred)
    print("Training Set Metrics:")
    print(f"  RMSE: {rmse_train:.4f}")
    print(f"  R2 Score: {r2_train:.4f}")
    print(f"  MAE: {mae_train:.4f}")

    # Evaluate on Testing Set
    y_test_pred = model.predict(X_test_data)
    rmse_test = np.sqrt(mean_squared_error(y_test_data, y_test_pred))
    r2_test = r2_score(y_test_data, y_test_pred)
    mae_test = mean_absolute_error(y_test_data, y_test_pred)
    print("Testing Set Metrics:")
    print(f"  RMSE: {rmse_test:.4f}")
    print(f"  R2 Score: {r2_test:.4f}")
    print(f"  MAE: {mae_test:.4f}")
    return (rmse_train, r2_train, mae_train), (rmse_test, r2_test, mae_test)

# 1. Linear Regression
linear_reg_model = LinearRegression()
train_evaluate_model(linear_reg_model, "Linear Regression", X_train_scaled, y_train, X_test_scaled, y_test)

# 2. Decision Tree Regressor with generalized hyperparameters
# Limiting max_depth and ensuring a minimum number of samples per leaf to reduce overfitting
decision_tree_model = DecisionTreeRegressor(max_depth=10, min_samples_leaf=5, random_state=42)
train_evaluate_model(decision_tree_model, "Decision Tree Regressor", X_train_scaled, y_train, X_test_scaled, y_test)

# 3. Random Forest Regressor with generalized hyperparameters
# Fewer estimators and limited depth to reduce overfitting, also min_samples_leaf
random_forest_model = RandomForestRegressor(n_estimators=100, max_depth=15, min_samples_leaf=5, random_state=42, n_jobs=-1)
train_evaluate_model(random_forest_model, "Random Forest Regressor", X_train_scaled, y_train, X_test_scaled, y_test)

# 4. XGBoost Regressor with generalized hyperparameters
# Limiting max_depth, reducing learning_rate, and adding subsample/colsample_bytree for generalization
xgb_model = XGBRegressor(objective='reg:squarederror', n_estimators=100, learning_rate=0.1, max_depth=5,
                         subsample=0.8, colsample_bytree=0.8, random_state=42, n_jobs=-1)
train_evaluate_model(xgb_model, "XGBoost Regressor", X_train_scaled, y_train, X_test_scaled, y_test)
```

```
Datasets loaded successfully.

--- Linear Regression ---
Training Set Metrics:
```

```
  RMSE: 42.0373
  R2 Score: 0.8129
  MAE: 30.6736
Testing Set Metrics:
  RMSE: 42.0272
  R2 Score: 0.8128
  MAE: 30.6093

--- Decision Tree Regressor ---
Training Set Metrics:
  RMSE: 2.0598
  R2 Score: 0.9996
  MAE: 0.2966
Testing Set Metrics:
  RMSE: 2.3540
  R2 Score: 0.9994
  MAE: 0.3404

--- Random Forest Regressor ---
Training Set Metrics:
  RMSE: 1.5153
  R2 Score: 0.9998
  MAE: 0.1316
Testing Set Metrics:
  RMSE: 2.4009
  R2 Score: 0.9994
  MAE: 0.2176

--- XGBoost Regressor ---
Training Set Metrics:
  RMSE: 4.1043
  R2 Score: 0.9982
  MAE: 2.1091
Testing Set Metrics:
  RMSE: 4.1631
  R2 Score: 0.9982
  MAE: 2.0959
((np.float64(4.104265153946603), 0.9982167082655299, 2.1091384149123673),
 (np.float64(4.163086141326116), 0.998163039361828, 2.0959069129547143))
```

```
from sklearn.model_selection import GridSearchCV

# Re-using the train_evaluate_model function from the previous cell
# Make sure the previous cell (UmU0TfYOUnXg) has been run to define X_train_scaled, y_train, X_test_scaled, y_test
# and the train_evaluate_model function
```

```
# and the train_evaluate_model function.

# --- Decision Tree Regressor Hyperparameter Tuning ---
print("\n--- Decision Tree Regressor Hyperparameter Tuning ---")
dt_param_grid = {
    'max_depth': [5, 10, 15, 20],
    'min_samples_leaf': [1, 2, 4, 8],
    'min_samples_split': [2, 5, 10]
}
dt_regressor = DecisionTreeRegressor(random_state=42)
dt_grid_search = GridSearchCV(estimator=dt_regressor, param_grid=dt_param_grid,
                              cv=5, n_jobs=-1, verbose=1, scoring='r2')
dt_grid_search.fit(X_train_scaled, y_train)

print(f"Best parameters for Decision Tree: {dt_grid_search.best_params_}")
print(f"Best R2 score on validation set for Decision Tree: {dt_grid_search.best_score_:.4f}")

best_dt_model = dt_grid_search.best_estimator_
print("\nEvaluating Best Decision Tree Model on full dataset:")
train_evaluate_model(best_dt_model, "Optimized Decision Tree Regressor", X_train_scaled, y_train, X_test_scaled, y_test)

# --- Random Forest Regressor Hyperparameter Tuning ---
print("\n--- Random Forest Regressor Hyperparameter Tuning ---")
rf_param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [10, 20, None], # None means unlimited depth
    'min_samples_leaf': [1, 2, 4]
}
rf_regressor = RandomForestRegressor(random_state=42, n_jobs=-1)
rf_grid_search = GridSearchCV(estimator=rf_regressor, param_grid=rf_param_grid,
                              cv=3, n_jobs=-1, verbose=1, scoring='r2') # Reduced CV folds for faster execution
rf_grid_search.fit(X_train_scaled, y_train)

print(f"Best parameters for Random Forest: {rf_grid_search.best_params_}")
print(f"Best R2 score on validation set for Random Forest: {rf_grid_search.best_score_:.4f}")

best_rf_model = rf_grid_search.best_estimator_
print("\nEvaluating Best Random Forest Model on full dataset:")
train_evaluate_model(best_rf_model, "Optimized Random Forest Regressor", X_train_scaled, y_train, X_test_scaled, y_test)
```

```
--- Decision Tree Regressor Hyperparameter Tuning ---
Fitting 5 folds for each of 48 candidates, totalling 240 fits
Best parameters for Decision Tree: {'max_depth': 15, 'min_samples_leaf': 4, 'min_samples_split': 10}
Best R2 score on validation set for Decision Tree: 0.9996

Evaluating Best Decision Tree Model on full dataset:

--- Optimized Decision Tree Regressor ---
Training Set Metrics:
  RMSE: 1.1897
  R2 Score: 0.9999
  MAE: 0.0882
Testing Set Metrics:
  RMSE: 2.0519
  R2 Score: 0.9996
  MAE: 0.1646

--- Random Forest Regressor Hyperparameter Tuning ---
Fitting 3 folds for each of 27 candidates, totalling 81 fits
Best parameters for Random Forest: {'max_depth': 20, 'min_samples_leaf': 1, 'n_estimators': 50}
Best R2 score on validation set for Random Forest: 0.9997

Evaluating Best Random Forest Model on full dataset:

--- Optimized Random Forest Regressor ---
Training Set Metrics:
  RMSE: 0.6041
  R2 Score: 1.0000
  MAE: 0.0514
Testing Set Metrics:
  RMSE: 1.7545
  R2 Score: 0.9997
  MAE: 0.1541
((np.float64(0.6040861503985542), 0.9999613678095124, 0.05135302132868041),
 (np.float64(1.7545232216797118), 0.9996737223714046, 0.1540536084586856))
```